Allineamento della produzione — ticket CRF (versione 27.1.0, aggiornata al 25/10/2026)

Guida passo-passo, in ordine, per portare la produzione allo stato di staging. Sostituisce la sequenza della guida del 22/10 (2026-10-22-guida-installazione-produzione-ticket-crf.md) e include le novità di 24–25/10: visibilità per gruppo, segno di chiusura firmato, ruolo admin del centro, correzioni della revisione di sicurezza.

Principi - Tutto si installa con il blocco **spento** (crf.confirm.require-closed-tickets=false); il blocco si accende solo alla fine (passo 10). Spento, nulla cambia per gli utenti. - Nessun segreto nel repository o in questo file: token e password si prendono dal gestore delle credenziali e si scrivono solo nel .env del server. - Mai tag a mano e mai commit su develop/master: rilasci e deploy passano da Jenkins. - Un passo alla volta; se un controllo di un passo non è verde, ci si ferma.

Legenda: **[TU]** = serve una persona con i permessi (Zammad, Keycloak admin, Jenkins); **[CLAUDE]** = posso prepararlo o eseguirlo io con il tuo ok.

Fase A — Prima di toccare la produzione

Passo 1. Decisioni e prerequisiti [TU]

Cosa	Scelta consigliata
Quali gruppi chiudono i ticket (internal:close:ticket)	Admins del centro + Farmacovigilanza (come staging)
Quali gruppi confermano le CRF (internal:confirm:crf)	Admins del centro (come staging)
Chi ha la deroga (internal:override:ticket-gate)	Pochi responsabili: servono i nomi dei gruppi
Chi sono gli admin del centro (internal:manage:center-tickets)	I gruppi Admins di **ogni** centro
Deroga con Zammad non verificabile (crf.confirm.override-when-unverifiable)	false
Antivirus allegati	Rimandato (nessuna scansione): il DPO deve esserne informato

Cosa	Scelta consigliata
Ticket vecchi chiusi prima del rilascio	Si convalidano dal pannello o con una bonifica concordata (passo 12)

Passo 2. Verifiche di capacità su Zammad di produzione [TU, sola lettura]

1. Disco sotto l'**85%** sul server di Zammad e indici di Elasticsearch **verdi o gialli** (non rossi): curl -s localhost:9200/_cluster/health. Con il disco sopra il 90% la ricerca di Zammad restituisce sempre vuoto e nessun utente risulta collegato (errore 403 «Utente non presente su Zammad»). Non alzare le soglie di Elasticsearch come su staging.
2. GET /api/v1/ticket_states: deve esistere lo stato **7 «annuncio studio»** (tipo *pending reminder*), usato dalle news di studio. Uno stato di tipo sconosciuto **blocca** la conferma.
3. Utente di servizio **dedicato** su Zammad (non un utente reale, non quello personale) con ruoli Admin e Agent: sarà il proprietario del token. Un utente reale verrebbe riscritto dal sync o disattivato come «orfano».

Passo 3. Backup [TU]

Prima di qualsiasi modifica: .env e httpd-config/versions.json; realm Keycloak (export); volumi postgres_data_hapi, postgres_data_keycloak, database di Zammad. Annotare le versioni attuali di ogni immagine per il rollback.

Fase B — Rilascio del software

Passo 4. Rilasciare irccs-common 27.1.0 [TU, Jenkins]

Le PR dei microservizi restano rosse finché common 27.1.0 non esiste. Ordine: 1. PR irccs-common (feature/PASTRL-720/ticket-crf-zammad-live) → develop → master, poi il job **irccs-release** (crea il tag yyyy-mm; non si tagga a mano). 2. Verificare che la versione **27.1.0** sia su Nexus.

Passo 5. Rilasciare microservizi, dashboard e irccs-docker [TU, Jenkins]

Stesso branch in ogni repo, stessa strada: PR → develop → master. 1. **Microservizi con codice nuovo:** irccs-microservice-zammad, irccs-microservice-studio-clinico, irccs-microservice-anagrafica-pazienti. 2. **Gli altri 8 microservizi:** solo bump di versione a 27.1.0 (la CI li ricostruisce). 3. **irccs-react-dashboard**: deve includere anche l'SSO

(feature/zammad-sso-auth-sso), come la dashboard di staging. 4. **irccs-docker**: script, configurazioni e documentazione.

L'ultimo giro di test su staging è stato fatto su immagini costruite a mano: in produzione si usano **solo** le immagini prodotte dalla CI.

Fase C — Zammad e Keycloak (con blocco spento)

Passo 6. Token Zammad nuovo [TU]

1. In Zammad, sull'utente di servizio del passo 2, crea un **token nuovo**.
2. Scrivilo solo nel .env del server di produzione come ZAMMAD_TOKEN_SERVICE=….
3. **Revoca il token vecchio**: quello nel repository (nella storia git del ms zammad e nelle immagini già pubblicate) va considerato compromesso. Controllo: con il token vecchio, GET /api/v1/users/me deve rispondere 401.
4. Il token deve coprire tutti i gruppi Zammad usati dalla piattaforma. Dal ms 27.1.0 il sync li apre da solo; l'utente di servizio deve avere il ruolo Admin.

Passo 7. Attributi in Zammad [TU o CLAUDE]

```
cd irccs-docker/setup/install
bash install_zammad_attributes.sh
```

Crea gli attributi crfscope, qrid, crfclosure e keycloak_ref (su utenti, organizzazioni, gruppi) e nasconde in modifica i campi patient, careplan, crf, crfscope, qrid, crfclosure. Poi Zammad si riavvia da solo: attendere.

Passo 8. Ruoli e risorse in Keycloak [TU o CLAUDE]

Dalla cartella irccs-docker/keycloak-config/, con le credenziali admin **solo da ambiente** (nessun default: lo script si ferma se ADMIN_PASS manca):

```
KC_URL=https://<host>/backoffice REALM=<realm> ADMIN_USER=<admin>
ADMIN_PASS=<da gestore credenziali> \
python3 add-ticket-gate-roles.py --dry-run     # controlla cosa verrebbe creato
python3 add-ticket-gate-roles.py
python3 add-zammad-authz.py --dry-run
python3 add-zammad-authz.py
```

Ruoli creati: internal:confirm:crf, internal:close:ticket, internal:override:ticket-gate, **internal:manage:center-tickets** (nuovo).

In alternativa, per tutti i passi 7–8 e i controlli di configurazione: setup/install/align_ticket_crf.sh (opzioni --from N, --only N, --yes), che chiede conferma prima di ogni scrittura.

Passo 9. Assegnare i ruoli ai gruppi [TU]

Dashboard → **Gruppi utenti** → sezione CRF (oppure Keycloak → Groups → Role mapping):

Ruolo	A chi
internal:confirm:crf	Gruppi Admins del centro
internal:close:ticket	Admins del centro e Farmacovigilanza
internal:override:ticket-gate	Pochi responsabili
internal:manage:center-tickets	Gruppi **Admins di ogni centro** (vedono e gestiscono i ticket di tutti i gruppi del centro)

Dopo l'assegnazione gli utenti devono rifare il **login**.

Fase D — Configurazione e deploy

Passo 10. .env e configurazione del ms zammad [TU o CLAUDE]

Nel .env di produzione (esempio in .env_example):

```
ZAMMAD_BASE_URL=http://irccs-zammad-nginx:8080/api/v1/     # alias senza underscore, non il nome del container Zammad
ZAMMAD_TOKEN_SERVICE=<token del passo 6>
ZAMMAD_CLOSURE_HMAC_SECRET=<nuovo segreto>                 # openssl rand -base64 48, generato sul server, mai in git
CRF_CONFIRM_REQUIRE_CLOSED_TICKETS=false                   # resta false fino al passo 14
CRF_CONFIRM_OVERRIDE_WHEN_UNVERIFIABLE=false
ZAMMAD_TICKET_LOOKUP_TIMEOUT=3s
CRF_CONFIRM_TICKET_LOOKUP_TIMEOUT=5s                       # deve restare MAGGIORE del precedente
ZAMMAD_TICKET_LOOKUP_COVERAGE_CHECK=true
ZAMMAD_TICKET_LOOKUP_COVERAGE_IGNORE_GROUPS=Users
```

Nel file montato microservice-config-zammad/application.properties devono esserci:

zammad.token=${ZAMMAD_TOKEN_SERVICE}
zammad.closure.hmac-secret=${ZAMMAD_CLOSURE_HMAC_SECRET:}
quarkus.http.limits.max-body-size=45M # servono per le news di studio con allegati

Se la prod usa ancora la versione precedente (webhook e copia in HAPI), togliere quarkus.keycloak.policy-enforcer.paths.7.* e le variabili ZAMMAD_TICKET_REF_*, ZAMMAD_WEBHOOK_SECRET, ZAMMAD_INSTANCE_ID.

> **Attenzione al segreto HMAC:** con il segreto impostato i ticket chiusi **prima** di questa versione risultano «chiusi senza convalida» e bloccano la conferma finché non vengono convalidati (passo 16). Cambiare il segreto in seguito invalida tutti i segni già scritti.

Passo 11. Deploy delle immagini [TU, Jenkins]

Job Deploy-Preprod-Jenkinsfile (poi produzione), un servizio alla volta, **riavviando pochi container per volta** (il carico sale molto al riavvio di molti servizi). Ordine consigliato: ms zammad, studio-clinico, anagrafica-pazienti, gli altri ms, dashboard/httpd. Il job rigenera versions.json. Se si deploya a mano: bash httpd-config/generate-versions-env.sh .env httpd-config/versions.json e poi docker restart irccs-httpd-dashboard, altrimenti la pagina Versioni mostra dati vecchi.

Fase E — Verifiche (blocco ancora spento)

Passo 12. Controlli sui log e sulla configurazione [TU o CLAUDE]

1. Sync Keycloak→Zammad completato: nei log del ms zammad «Keycloak to Zammad sync completed», senza errori; gli utenti hanno keycloak_ref.
2. Copertura gruppi: «N gruppi Zammad aperti in full al token di servizio» e nessun «non aperti».
3. **Nessun** avviso «zammad.closure.hmac-secret non impostato».
4. Dal container del ms zammad: curl -s -o /dev/null -w '%{http_code}' http://irccs-zammad:8080/q/health → 200; da studio-clinico: l'indirizzo in crf.confirm.ticket-service-url risponde.
5. GET /fhir/zammad/tickets/open-for-gate?patient=<id> con il service account → 200; con un utente normale → 403.

Passo 13. Prove funzionali su un paziente di prova [TU o CLAUDE]

Mai su pazienti reali: usare un **paziente sintetico** con una CRF di prova. Con utenti reali nei tre gruppi: 1. Sperimentatore apre un ticket a Farmacovigilanza:

lo vedono sperimentatore (chi l'ha aperto), Farmacovigilanza e admin del centro; **non** lo vede un altro sperimentatore. 2. Farmacovigilanza risponde e chiude con una motivazione; il ticket risulta chiuso con il segno firmato. 3. Lo sperimentatore non può chiudere (403) né confermare la CRF (403). 4. Con blocco ancora spento nessuna conferma cambia comportamento. Gli script E2E in setup/test/ (e2e_ticket_workflow.py, e2e_ticket_visibility.py, e2e_ticket_roles.py) sono pensati per staging: richiedono E2E_PATIENT ed E2E_CAREPLAN (nessun default), creano utenti Keycloak temporanei e un riavvio del ms zammad. In produzione usarli solo con un paziente sintetico e dopo un OK esplicito.

Fase F — Attivazione

Passo 14. Accendere il blocco [TU]

1. CRF_CONFIRM_REQUIRE_CLOSED_TICKETS=true nel .env; ricreare **studio-clinico** e il **ms zammad** (entrambi leggono il flag).
2. Prove sul paziente di prova:
   - CRF con ticket aperto → conferma **409 CRF_OPEN_TICKETS**;
   - ticket chiuso dalla piattaforma → conferma **200**;
   - ticket chiuso a mano in Zammad → ancora **409**;
   - deroga con motivazione ≥10 caratteri (e il ruolo di deroga) → conferma **200** con audit;
   - Zammad fermo → **503 CRF_TICKET_STATE_UNVERIFIABLE.**
3. Se compare il 503 subito: controllare nell'ordine l'indirizzo crf.confirm.ticket-service-url, la validità del token e i timeout. **Per tornare operativi: flag a false e riavvio.**

Passo 15. Monitoraggio [TU]

Per qualche giorno controllare su Loki/Grafana i 409 (attesi) e i 503 (anomali: Zammad lento o giù).

Fase G — Dopo l’attivazione

Passo 16. Ticket chiusi prima del rilascio [TU o CLAUDE]

Risultano «chiusi senza convalida» e bloccano. Si convalidano uno a uno dal pannello (motivazione obbligatoria) oppure con una bonifica concordata col cliente: posso prepararne l’elenco, in sola lettura.

Passo 17. Comunicazioni e documentazione

- Avvisare gli utenti che i diritti sono cambiati e che serve un **nuovo login**.
- Consegnare al cliente il documento funzionale (2026-10-25-ticket-crf-documento-funzionale-cliente) e portare le modifiche nel repo di produzione.
- Segnalare al DPO: dati sanitari nei ticket, nessun antivirus sugli allegati, motivazioni di deroga come testo libero in audit.

Rollback

1. **Subito:** CRF_CONFIRM_REQUIRE_CLOSED_TICKETS=false e riavvio di studio-clinico e ms zammad. Nessun dato da toccare.
2. **Versione precedente:** ridistribuire le immagini annotate al passo 3. Ruoli Keycloak e attributi Zammad possono restare.
3. Il token Zammad vecchio **non** va ripristinato: è compromesso.

Riepilogo dell’ordine

Decisioni (1) → capacità Zammad (2) → backup (3) → common (4) → altri repo (5) → token (6) → attributi Zammad (7) → ruoli e risorse Keycloak (8) → assegnazione ruoli (9) → .env e config (10) → deploy (11) → verifiche (12–13) → accensione (14) → monitoraggio (15) → bonifica ticket vecchi (16) → comunicazioni (17).